

## # Lambda functions

A lambda function is a small, anonymous function without a name.

- It is used when
- ① logic is short
  - ② you need a function only once
  - ③ writing def feels overkill

### Normal vs Lambda functions

#### Normal

```
def add(a,b):  
    return a+b  
print(add(2,3))
```

#### Lambda

```
add = lambda a,b: a+b  
print(add(2,3))
```

### Syntax of Lambda

Lambda arguments: expression.

### Important rule

- Can have multiple arguments
- Can have only one expression
- Cannot use loops (for, while)
- Cannot use multiple line
- Return value automatically

Exercise 21

## Expense Tracker & Analyzer.

- increases expenses due to tax.
- filter high expense
- calculate total spending
- find maximum expense.

[take Expenses from user]

↓

[convert all Expenses into a list]

↓

lambda function ← [increase Expense By 18% due to tax]

↓

filter() ← [filter Expense > 5K]

↓

reduce() ← [calculate total spending]

↓

[using if else  
find max Expense]



DATE \_\_\_\_\_  
PAGE \_\_\_\_\_

lambda with map(), filter() and reduce()

① Map() → it apply function to each items

```
num = [1, 2, 3, 4, 5]
result = list (map(lambda x: x*2, num))
print(result)
```

if we don't  
use list keyword  
it give us a map.

it gives us a list  
[1, 4, 6, 8, 10]

② filter() - filter each items

```
num = [1, 2, 3, 4, 5]
result = list (filter(lambda x: x%2 == 0, num))
print(result)
```

# [2, 4]

③ reduce() - reduce to single value.

from functools import reduce

```
num = [1, 2, 3, 4]
```

~~result = list()~~

```
result = reduce(lambda a, b: a+b, num)
```

# [10]

now?

[1, 2, 3, 4]  
1+2 = 3

[3, 3, 4]  
3+3 = 6

[6, 4] → [10]